
HERRAMIENTAS AVANZADAS:

CNNs 2D y PyBullet

INTRODUCCIÓN

En la actualidad se dispone de varios modelos de IA para resolver el problema de detección y seguimiento de objetos. En el segundo proyecto de esta asignatura se utilizarán dos cámaras, por ello se deberá detectar y seguir puntos de interés de los objetos reales en los pares de imágenes capturados por el sistema de estereovisión y resolver el problema de correspondencias. Inicialmente se recomienda resolver los problemas de detección y correspondencias de manera manual, marcando con el ratón la posición de los puntos de interés (o centros de las regiones de interés) de cada objeto sobre las distintas imágenes. Después, resolver automáticamente los problemas de detección y correspondencias de los puntos de interés de todos los objetos utilizando funcionalidad de OpenCV, Yolo, MediaPipe o ZED2.

Por otra parte, en el segundo proyecto de la asignatura también se deberá crear y mostrar una escena virtual más compleja (que sea un gemelo digital de la escena de la aplicación) y un visor que muestre la escena utilizando la funcionalidad de open3D o de un simulador de robótica interactivo, como por ejemplo PyBullet. Esta escena virtual debe incluir al menos un objeto real y un robot virtual.

OBJETIVOS

El objetivo principal de esta sesión de prácticas es aprender a utilizar modelos IA para resolver el problema de detección y seguimiento de objetos. Para ello, se utilizarán diversos modelos Yolo, MediaPipe o ZED2. Además, se presentarán las bases fundamentales para aprender a utilizar el simulador PyBullet.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/90-Jupyter lab demo – CNNs 2D Trackers
- PoliformaT -> Recursos/2-Software-&-cuadernos/91-ZED2 sample – CNNs 2D Trackers
- PoliformaT -> Recursos/2-Software-&-cuadernos/92-Jupyter lab demo – CNNs 2D Body tracking
- PoliformaT -> Recursos/2-Software-&-cuadernos/93-Jupyter lab demo – PyBullet

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

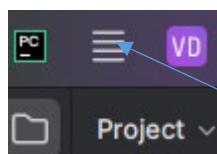
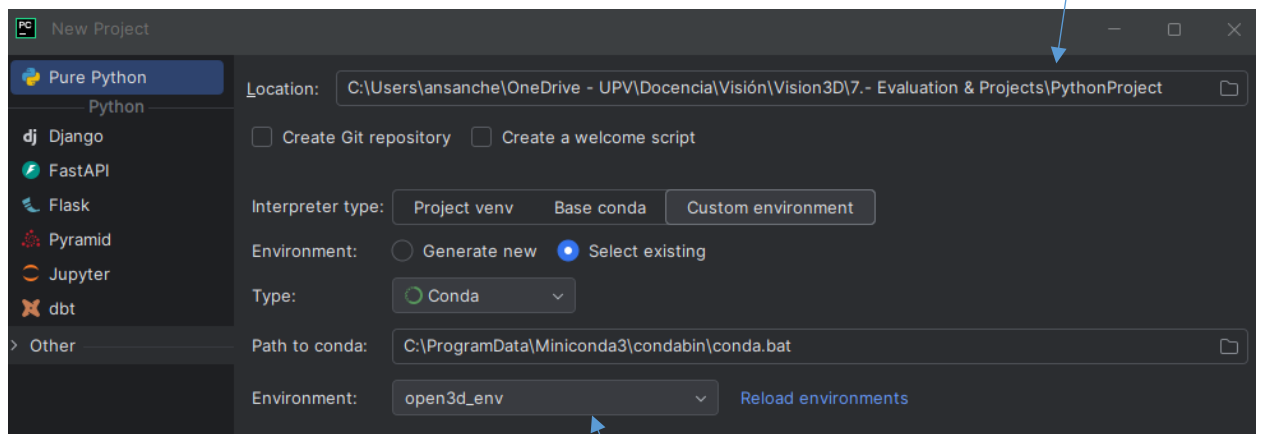
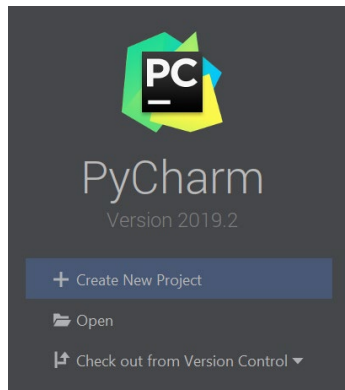
```
(base) C:\windows\system32> conda activate open3d_env
```

```
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working\_folder\_path”
```

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.



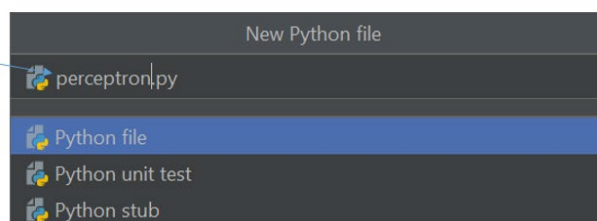
Main menu

Select environment

Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



EJERCICIO 1: DETECCIÓN Y SEGUIMIENTO DE OBJETOS CON YOLO

```
import cv2
import sys
import numpy as np
from ultralytics import YOLO
from collections import defaultdict

# Load the YOLO11 model
model = YOLO("yolo11n.pt")
#model = YOLO("yolo11n-seg.pt")

# Open the video file
cap = cv2.VideoCapture(0)

# Store the track history
track_history = defaultdict(lambda: [])

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO11 tracking on the frame, persisting tracks between frames
        results = model.track(frame, persist=True)

        # Get the boxes and track IDs
        boxes = results[0].boxes.xywh.cpu()
        track_ids = results[0].boxes.id.int().cpu().tolist()

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Plot the tracks
        for box, track_id in zip(boxes, track_ids):
            x, y, w, h = box
            track = track_history[track_id]
            track.append((float(x), float(y))) # x, y center point
            if len(track) > 30: # retain 90 tracks for 90 frames
                track.pop(0)

            # Draw the tracking lines
            points = np.hstack(track).astype(np.int32).reshape((-1, 1, 2))
            cv2.polylines(annotated_frame, [points], isClosed=False, color=(230, 230, 230), thickness=10)

        # Display the annotated frame
        cv2.imshow("YOLO Tracking", annotated_frame)

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break

# Release the video capture object and close the display window
cap.release()
cv2.destroyAllWindows()
```

Ultralytics YOLO dispone de [funciones de detección y seguimiento](#) de objetos robustas y versátiles.

Ultralytics YOLO dispone de los siguientes algoritmos de seguimiento. Pueden activarse pasando el archivo de configuración YAML correspondiente, como por ejemplo `tracker=tracker_type.yaml`:

- [BoT-SORT](#) - Utilización `botsort.yaml` para activar este rastreador.
- [ByteTrack](#) - Utilización `bytetrack.yaml` para activar este rastreador.

El seguidor por defecto es BoT-SORT, pero se puede utilizar ByteTrack con:

```
results = model.track(frame, ..., tracker="bytetrack.yaml")
```

EJERCICIO 2: DETECCIÓN DE OBJETOS CON MEDIAPIPE

MediaPipe no está instalado en el entorno conda “open3d_env”, por ello para poder utilizarlo se deberá instalar previamente.

```
import cv2
import numpy as np

MARGIN = 10 # pixels
ROW_SIZE = 10 # pixels
FONT_SIZE = 1
FONT_THICKNESS = 1
TEXT_COLOR = (255, 0, 0) # red

def visualize(
    image,
    detection_result
) -> np.ndarray:
    """Draws bounding boxes on the input image and return it.
    Args:
        image: The input RGB image.
        detection_result: The list of all "Detection" entities to be visualize.
    Returns:
        Image with bounding boxes.
    """
    for detection in detection_result.detections:
        # Draw bounding_box
        bbox = detection.bounding_box
        start_point = bbox.origin_x, bbox.origin_y
        end_point = bbox.origin_x + bbox.width, bbox.origin_y + bbox.height
        cv2.rectangle(image, start_point, end_point, TEXT_COLOR, 3)

        # Draw label and score
        category = detection.categories[0]
        category_name = category.category_name
        probability = round(category.score, 2)
        result_text = category_name + ' (' + str(probability) + ')'
        text_location = (MARGIN + bbox.origin_x,
                        MARGIN + ROW_SIZE + bbox.origin_y)
        cv2.putText(image, result_text, text_location, cv2.FONT_HERSHEY_PLAIN,
                    FONT_SIZE, TEXT_COLOR, FONT_THICKNESS)

    return image

IMAGE_FILE = 'imgs/cat_and_dog.jpg'
img = cv2.imread(IMAGE_FILE)
cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

# STEP 1: Import the necessary modules.
import numpy as np
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision

# STEP 2: Create an ObjectDetector object.
base_options = python.BaseOptions(model_asset_path='efficientdet.tflite')
options = vision.ObjectDetectorOptions(base_options=base_options,
                                       score_threshold=0.5)
detector = vision.ObjectDetector.create_from_options(options)

# STEP 3: Load the input image.
image = mp.Image.create_from_file(IMAGE_FILE)

# STEP 4: Detect objects in the input image.
detection_result = detector.detect(image)

# STEP 5: Process the detection result. In this case, visualize it.
image_copy = np.copy(image.numpy_view())
annotated_image = visualize(image_copy, detection_result)
rgb_annotated_image = cv2.cvtColor(annotated_image, cv2.COLOR_BGR2RGB)
cv2.imshow("Image", rgb_annotated_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

EJERCICIO 3: SIMULACIÓN CON PYBULLET

Se dispone de información adicional en la carpeta:

- PoliformaT -> Recursos/2-Software-&-cuadernos/S1-pyBullet

En la siguiente función se crea una escena virtual que contiene un plano del suelo con la textura de un estadio, una portería, una pelota roja, un balón, un brazo robot y un robot móvil. En la demo se puede analizar cómo se pueden simular los movimientos de estos objetos virtuales.

```
def create_pyBullet_scene():

    p.connect(p.GUI) # Inicializa la escena del simulador pyBullet
    p.setGravity(0, 0, -9.8) # Configura la gravedad
    p.setAdditionalSearchPath(pybullet.data.getDataPath()) # Habilita el directorio data
    p.resetDebugVisualizerCamera(cameraDistance=5.5, cameraYaw=60, cameraPitch=-30,
                                cameraTargetPosition=[0, 0, 0])

    escena = pyBulletVirtualWorld(p)

    plano_suelo = p.loadSDF("data/stadium.sdf", globalScaling=0.2665)
    q4 = p.getQuaternionFromEuler([0., 0., 1.5707])
    for part in plano_suelo:
        p.resetBasePositionAndOrientation(part, [0, 0, 0], q4)

    goal = p.loadURDF("data/goal.urdf", globalScaling=0.25, useFixedBase=True)
    q4 = p.getQuaternionFromEuler([1.5707, 0., 3.1416])
    p.resetBasePositionAndOrientation(goal, [0, 7.25, 0], q4) # localización inicial

    py_pelota_height = 0.2
    radius_pelota = py_pelota_height / 2.
    py_pelota = p.createMultiBody(
        baseMass=0.1, # Masa de la pelota
        baseCollisionShapeIndex=p.createCollisionShape(shapeType=p.GEOM_SPHERE, radius=radius_pelota),
        baseVisualShapeIndex=p.createVisualShape(shapeType=p.GEOM_SPHERE, rgbColor=[1, 0, 0, 1],
        radius=radius_pelota)
    )

    p.resetBasePositionAndOrientation(py_pelota, [0, 5, radius_pelota], [0, 0, 0, 1])
    p.changeDynamics(py_pelota, -1, restitution=0.7) # Coeficiente de restitución (bounciness)
    pelota=pyBulletObject(py_pelota,py_pelota_height,init_position=[0, 5, radius_pelota],obj_type="virtual")
    escena.add_object(pelota)

    py_balon_height = 0.2
    radius_balon = py_balon_height / 2.
    py_balon = p.loadURDF("soccerball.urdf", [0, 2, radius_balon], globalScaling=py_balon_height)
    balon = pyBulletObject(py_balon, py_balon_height, init_position=[0, 2, radius_balon], obj_type="real")
    escena.add_object(balon)

    py_robot = p.loadURDF("data/kuka_iiwa/model.urdf", [0, 0, 0], globalScaling=5.0, useFixedBase=True)
    p.changeDynamics(py_robot, -1, restitution=0.0)
    joint_angles = (0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00) # pi/2 = 1.5707
    set_joint_angles(p, py_robot, joint_angles)
    (_, _, py_robot_height) = p.getAABB(py_robot)[1]
    robot = pyBulletObject(py_robot, py_robot_height, init_position=[0, 0, 0], obj_type="robot")
    escena.add_object(robot)

    (position, orientation) = get_current_pose(p, py_robot, 6)
    try:
        joint_angles = calculate_ik(p, py_robot, 6, position, orientation)
        set_joint_angles(p, py_robot, joint_angles)
    except:
        print("Localización no alcanzable:", position, orientation)

    py_turtle = p.loadURDF("data/turtlebot.urdf", [0, 0, 0], globalScaling=0.5)
    q4 = p.getQuaternionFromEuler([0, 0, 0])
    p.resetBasePositionAndOrientation(py_turtle, [0, 7, 0], q4)
    turtle = pyBulletObject(py_turtle, py_robot_height, init_position=[0, 7, 0], obj_type="mobile_robot")
    escena.add_object(turtle)

    return p, escena
```

4.-TAREA

A partir de los materiales y ejemplos anteriores deberéis integrar en vuestro proyecto:

- un método de detección y seguimiento de objetos que se aplique sobre las dos secuencias de imágenes capturadas por las cámaras del par de estereovisión,
- además, deberéis integrar un robot sobre la escena virtual del segundo proyecto.